

框架前置-服务端语言Node.js（老版本）

1.Node.js模块：

模块系统是Node.js最基本也是最常用的，一般情况分为四类

- 原生模块
- 文件模块
- 第三方模块
- 自定义模块
 - 第一步创建模块（b.js）
 - `function FunA () {`
 - `return 'Tom';`
 - `}`
 - `module.exports = FunA;`
 - 第二部加载模块（a.js）
 - `var FunA = require ('./b.js');`
 - `var name = FunA ();`
 - `console.log (name);`
 - 功能介绍：module.exports
 - module.exports针对Node.js用于对外暴露，或者称对外开放指定访问权限的一个对象，上例中如果没有export代码，那么b文件中require为空，若一个模块中仅有一个module.exports，如果有多个后面的则会覆盖前面。
 - 功能介绍：exports
 - exports是module对象的属性，同时也是一个对象，很多时候一个js文件有多个需要暴露的方法，但是module.exports又只能暴露一个，这个时候要用到exports：

■ 创建模块：

- `function FunA () {`
- `return a;`
- `}`
- `function FunB () {`
- `return b;`
- `}`
- `exports.FunA = FunA;`
- `exports.FunB = FunB;`

■ 加载模块：将exports作为对象

- `var FunA = require ('./b.js');`
- `var name1 = FunA.FunA ();`
- `var name2 = FunA.FunB ();`
- `console.log (name1);`
- `console.log (name2);`

■ 加载模块：另一种方式，将exports作为属性

- `var {FunA,FunB} =require ('./b.js');`
- `var name1 = FunA ();`
- `var name2 = FunB ();`
- `console.log (name1);`
- `console.log (name2);`

2.NPM scripts

- npm脚本
 - npm允许在package.json文件里面，使用scripts字段定义脚本命令，package.json里面的scripts字段是一个对象，它的每一个属性对应一段脚本，定义在package.json里面的脚本，称之为npm脚本
 - 查看当前所有项目的所有npm脚本命令，可以使用不带参数的npm run命令

- npm run
- 使用npm脚本
 - npm run 脚本名称
 - 如果是并行执行脚本，使用&符号
 - npm run script1.js & npm run script2.js
 - 如果是继发执行，即前任务成功，执行下一个任务，使用&&符号
 - npm run script1.js && npm run scripts2.js
 - 简写形式
 - npm start即npm run start
 - npm stop即run
 - npm test即run
 - npm restart即npm run stop && npm run restart && npm run start
- npm 版本管理
 - 查看npm版本
 - npm -v
- nrm管理npm版本
 - npm install -g nrm
- 查看nrm可选的源有哪些，星号为当前使用源
 - nrm ls
- 切换nrm使用的源
 - nrm use taobao
- 增加定制的源，适用于添加企业内部私有源
 - nrm add <registry> <url>，源名+源路径
 - `nrm add registry http://registry.npm.frp.trmap.cn/`
- 删除对应的源
 - nrm del <registry>
- 测试源的速度

- nrm test npm

3.node部署

- forever
 - forever可以在cmd或ssh连接断开时，让项目一直运行，而且可以在项目崩溃时自动重启
 - 安装forever
 - npm install -g forever
 - 帮助手册
 - forever —help
 - 使用forever启动项目
 - forever start app.js
 - 使用forever停止项目
 - forever stop app.js
 - 列出所有通过forever管理的项目
 - forever list
 - 监视项目中的文件，当文件有变动时重启项目
 - forever -w start app.js
- pm2
 - 安装pm2
 - npm install pm2 -g
 - 新建运行脚本js，使用pm2运行
 - pm2 start index.js
 - 执行命令重启或暂停服务
 - pm2 stop name/id
 - pm2 restart name/id
 - pm2 delete name/id
 - 常用命令

- 运行: `pm2 start app.js`
- 查看运行状态: `pm2 list`
- 追踪资源运行情况: `pm2 monit`
- 查看日志: `pm2 logs`
- 重启应用: `pm2 restart name/id`
- 停止应用: `pm2 stop app.js`
- 开启api访问: `pm2 dashboard`
- 自动重启, 当文件发生改动, 则程序自动重启服务
 - `pm2 start app.js --watch`
 - 监控整个项目的文件, 如果只想监听指定文件和目录, 通过修改配置文件的 `watch`、`ignore_watch` 字段来设置
- 负载均衡
 - 命令如下, 表示开启三个进程, 如果 `-i 0`, 则会根据机器当前的核数自动开启尽可能多的进程
 - `pm2 start app.js -i 3` // 开启三个进程
 - `pm2 start app.js -i max` // 根据cpu核数, 开启对应数目的进程
- 日志查看
 - 除了可以打开日志文件查看日志外, 也可以通过 `pm2 logs` 来查看实时日志, 这点对于线上问题排查很重要,
 - 比如某个node服务突然异常重启, 可以通过日志工具查看实时日志, 是不是脚本出错导致的重启
 - `pm2 logs`
- 内存使用超过上限自动重启,
 - 如果想要应用, 在超过使用内存上线之后自动重启, 那么可以加上 `--max-memory-restart` 参数 (有对应的配置项)
 - `pm2 start big-array.js --max-memory-restart 20M`
- **pm2与forever对比**
 - keep alive (保持运行)
 - 确保应用程序在崩溃或者退出之后能够自动重启

- 两者都支持
- Coffeescript
 - 支持使用Coffeescript编写代码
 - forever支持支持，pm2可以通过配置文件和脚本简介支持
- Log aggregation（日志聚合）
 - 将所有的进程的日志集中管理，便于查看和分析
 - forever不支持，需要手动处理；pm2支持，使用pm2 logs命令
- API
 - 提供一个API接口，用于远程管理和监控进程
 - forever不支持，没有内置API接口；pm2支持，可以通过http请求或者命令行进行远程管理
- Terminal monitoring（终端监控）
 - 在终端中实时监控进程的状态和性能
 - forever不支持，监控能力有限；pm2支持，使用pm2 monit命令，实时监控进程CPU和内存使用情况
- Clustering（集群模式）
 - 支持在多核服务器上运行多个进程实例，以提高应用程序的性能和可用性
 - forever不支持，主要用于单进程管理；pm2支持，提供了集群模式cluster mode，可以自动管理多个进程
- JSON configuration（JSON配置文件）
 - 支持使用JSON文件来配置应用程序的启动参数和运行环境
 - 都支持，
- 总结：
- **forever**简单易用，功能有限，缺乏高级功能，适合个人开发、小型项目，简单的node.js应用
- **pm2**功能强大，支持日志聚合、API接口、集群模式、实时监控等高级功能，但是配置较为复杂，需要一定的学习成本，是何种大型项目、生产环境、高级管理功能的场景。

4.基础应用：

- 路由

在BS架构中，路由的概念是一样的，理解为根据客户端请求的URL映射到不同的方法实现，更多的是一般都是针对URL的路径，参数或者是锚点等信息进行映射。

- 场景

- 1.注册一个账号 → 【post】 → http: //localhost: 88/register
- 2.注册成功情况下跳转到登录界面进行登录 → 【post】 —>http: //localhost: 88.login
- 3.登陆成功进行获取学生信息 → 【get】 →http: //localhost: 88/students
- 4.同时也可以获取订单信息 → 【get】 →http: //localhost: 88/orders
- 5.如果访问不存在的路由，则抛出错误信息.

场景代码如下

```
const http = require('http')
const url = require('url')
const qs = require('querystring');
const util = require('util');

http.createServer((request, response) => {
  let urlObj = url.parse(request.url, true);
  let pathname = urlObj.pathname;
  let method = request.method.toUpperCase();
  let params = urlObj.query;
  if(method === 'POST'){
    let postData = '';
    request.on('data', (_data) => {
      postData += '_data';
    })
    request.on('end', () => {
```

```

postData = qs.parse(postData);
let result = {};
switch(pathname){
  case '/login':
    //连接数据库，实现登陆逻辑
    result = {status: true};
    break;
  case '/register':
    //连接数据库，实现注册逻辑
    result = {status: true};
    break;
  default :
    result = {status: false, message: '没有对
应的请求'};
    break;
}
response.end(util.inspect(result))
})
} else {
  let result = {};
  switch(pathname){
    case '/students':
      //连接数据库，获取学生信息
      result = {status: true, data: [], params};
      break;
    case '/orders':
      //连接数据库，获取订单信息
      result = {status: true, data: [], params};
      break;
    default :
      result = {status: false, message: '没有对应

```



```

    的请求', params};
        break;
    }
    response.end(util.inspect(result))
  }
}).listen(88)

```

- Demo即上述场景测试代码
- request

request是一个第三方的模块，可用于发起http或https请求，可以理解为服务端的ajax请求，可用于简单的服务器代理，用法和ajax类似，使用前安装该模块【npm install request —save】

- GET请求
 - 单参数使用方法

代码如下

```

request.get('https://cnodejs.org/api/v1/topics?page=1&limit=10', (error, response, body) => {
    console.log(body)
})
//or
request('https://cnodejs.org/api/v1/topics?page=1&limit=10', (error, response, body) => {
    console.log(body)
})

```

- 多参数设置

代码如下

- 使用 `request.get` 发送HTTP GET请求。

- `request.get` 的回调函数接收三个参数：
 - `err`：请求过程中发生的错误。
 - `res`：响应对象。
 - `body`：响应体（通常是字符串）。
- 如果发生错误（`err`），调用 `options.callback`，返回一个包含 `status: false` 和 `error` 的对象。
- 如果请求成功，根据响应状态码 `res.statusCode` 判断是否成功。如果状态码为 `200`，返回 `status: true`，否则返回 `status: false`。
- `options.callback` 的返回值是一个对象，包含 `status`、`error` 和 `data`。
- 如果在请求过程中捕获到异常，会记录到日志（假设 `logger.error` 是一个日志记录函数）。
- 如果 `try` 块中发生异常，会打印 `'http error'`。

```
exports.get = function(url, options) {
  options = options || {};
  var httpOptions = {
    url: url,
    method: 'get',
    timeout: options.timeout || 10000,
    headers: options.headers || default_post_headers,
    proxy: options.proxy || '',
    agentOptions: agentOptions,
    params: options.params || {}
  }
}
```

```

if(options.userAgent){
  httpOptions.headers = {
    'User-Agent': userAgents[options.userAgen
t],
  }
}

try{
  request.get(httpOptions, function(err, res, bod
y) {
    if (err) {
      options.callback({status: false, error: err})
    } else {
      options.callback({status: res.statusCode
== 200, error: res, data: body})
    }
  }).on('error', logger.error);
} catch(err){
  console.log('http error');
}
}

```

- POST请求

request支持application/x-www-form-urlencoded和multipart/form-data实现表单上传

- application/-x-www-form-urlencoded (URL-Encoded Forms)

代码示例如下

```

request.post('http://service.com/upload', {form:{ke
y:'value'}})

```

```
// or
request.post('http://service.com/upload').form({key: 'value'})
// or
request.post({url: 'http://service.com/upload', form: {key: 'value'}}, function(err, httpResponse, body) { /* ... */ })
```

- multipart/form-data (Multipart Form Uploads)

代码示例如下

```
var formData = {
  // Pass a simple key-value pair
  my_field: 'my_value',
  // Pass data via Buffers
  my_buffer: new Buffer([1, 2, 3]),
  // Pass data via Streams
  my_file: fs.createReadStream(__dirname + '/unicycle.jpg'),
  // Pass multiple values /w an Array
  attachments: [
    fs.createReadStream(__dirname + '/attachment1.jpg'),
    fs.createReadStream(__dirname + '/attachment2.jpg')
  ],
  // Pass optional meta-data with an 'options' object
  // with style: {value: DATA, options: OPTIONS}
  // Use case: for some types of streams, you'll need to provide "file"-related information manually.
```

```
// See the `form-data` README for more informati
on about options: https://github.com/form-data/for
m-data
custom_file: {
  value: fs.createReadStream('/dev/urandom'),
  options: {
    filename: 'topsecret.jpg',
    contentType: 'image/jpeg'
  }
}
};
request.post({url:'http://service.com/upload', formD
ata: formData}, function optionalCallback(err, httpR
esponse, body) {
  if (err) {
    return console.error('upload failed:', err);
  }
  console.log('Upload successful! Server responde
d with:', body);
});
```

- 常用多参数设置

代码示例如下

```
exports.form_post = function(url, postdata, options)
{
  // console.log(`${moment().format()} HttpFormPo
st: ${url}`)
  return new Promise((resolve, reject) => {
    options = options || {};
  });
}
```

```

var httpOptions = {
  url: url,
  form: postdata,
  method: 'post',
  timeout: options.timeout || 3000,
  headers: options.headers || default_post_headers,
  proxy: options.proxy || '',
  agentOptions: agentOptions
};
request(httpOptions, function(err, res, body) {
  if (err) {
    reject(err);
  } else {
    if (res.statusCode === 200) {
      resolve(body);
    } else {
      reject(res.statusCode);
    }
  }
}).on('error', logger.error);
});
};

```

- 流

流是处理数据的方式，允许逐块的读取或写入数据，而不是一次性整个数据加载到内存中，流可以有效地处理大量数据，尤其是当数据量超过内存限制时，流广泛应用于文件操作、网络请求、数据处理等场景

代码示例如下

```
request('http://img.zcool.cn/community/018d4e554967920000019ae9df1533.jpg@900w_1l_2o_100sh.jpg').pipe(fs.createWriteStream('test.png'))//下载图片并保存
request('https://cnodejs.org/api/v1/topics?page=1&limit=10').pipe(fs.createWriteStream('cnodejs.json'))//获取JSON数据并保存到指定文件
```

- http小爬虫

在服务端去请求外部的url拿到对方的资源，然后进行分析并抓取有效数据

代码示例如下

```
const request = require('request');
const fs = require('fs');
const cheerio = require('cheerio');

request('http://www.lanrentuku.com/', (error, response, body) => {
  let $ = cheerio.load(body);
  $('img', '.in-ne').each((i, e) => {
    let src = $(e).attr('src');
    let name = src.substr(src.lastIndexOf('/') + 1);
    request(src).pipe(fs.createWriteStream(name))
  })
})
```

- async

node.js是一个异步机制的服务端语言，在大量异步的场景下需要按顺序执行，正常做法就是回调嵌套回调，然而回调嵌套太多会很麻烦，之后便退出了异步控制流—

Async

- Async/Await

Async/await就ES7的方案结合ES6的promise对象，主要作用是避免回调多层嵌套，以最接近同步代码的方式编写异步代码

- 基本规则

- async表示这是一个async异步函数，await只能用在异步函数里面
- await表示等待到promise返回结果，再继续执行代码
- await后面应该是一个promise对象

- 对比使用

3秒后返回值

- 原始代码

```
let sleep = (time, cb) => {
  setTimeout(() => {
    cb('ok');
  }, 3000);
}

let start = () => {
  sleep(3000, (result) => {
    console.log(result)
  })
}

start()
```

- promise代码

```
let sleep = (time) => {
  return new Promise((resolve, reject) => {
```



```

    setTimeout(() => {
      resolve('ok') ;
    }, time);
  })
}

let start = () => {
  sleep(3000).then((result) => {
    console.log(result)
  })
}

start()

```

- aysnc/await代码

```

let sleep = (time) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve('ok') ;
    }, time);
  })
}

let start = async () => {
  let result = await sleep(3000);
  console.log(result)
}

```

```
start();
```

- 捕捉错误

```
let sleep = (time) => {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      reject('error') ;  
    }, time);  
  })  
}
```

```
let start = async () => {  
  try{  
    let result = await sleep(3000);  
    console.log(result)  
  } catch(err) {  
    console.log('error')  
  }  
}
```

```
start();
```

- 在循环中使用

```
let sleep = (time) => {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve('ok') ;  
    }, time);  
  })  
}
```

```

    }, time);
  })
}

let start = async () => {
  for (var i = 1; i <= 3; i++) {
    console.log(`当前是第${i}次等待..`);
    await sleep(1000);
  }
}

start();

```

- 爬虫中使用

```

const request = require('request');
const fs = require('fs');
const cheerio = require('cheerio');

let spider = (url) => {
  return new Promise((resolve, reject) => {
    request(url, (error, response, body) => {
      resolve(body);
    })
  })
}

let start = async () => {
  let dom = await spider('http://www.lanrentuku.com/');

```

```
let $ = cheerio.load(dom);
$('img', '.in-ne').each((i, e) => {
  let src = $(e).attr('src');
  let name = src.substr(src.lastIndexOf('/') + 1);
  request(src).pipe(fs.createWriteStream(name))
})
}

start();
```